

Siyanda Ndlovu

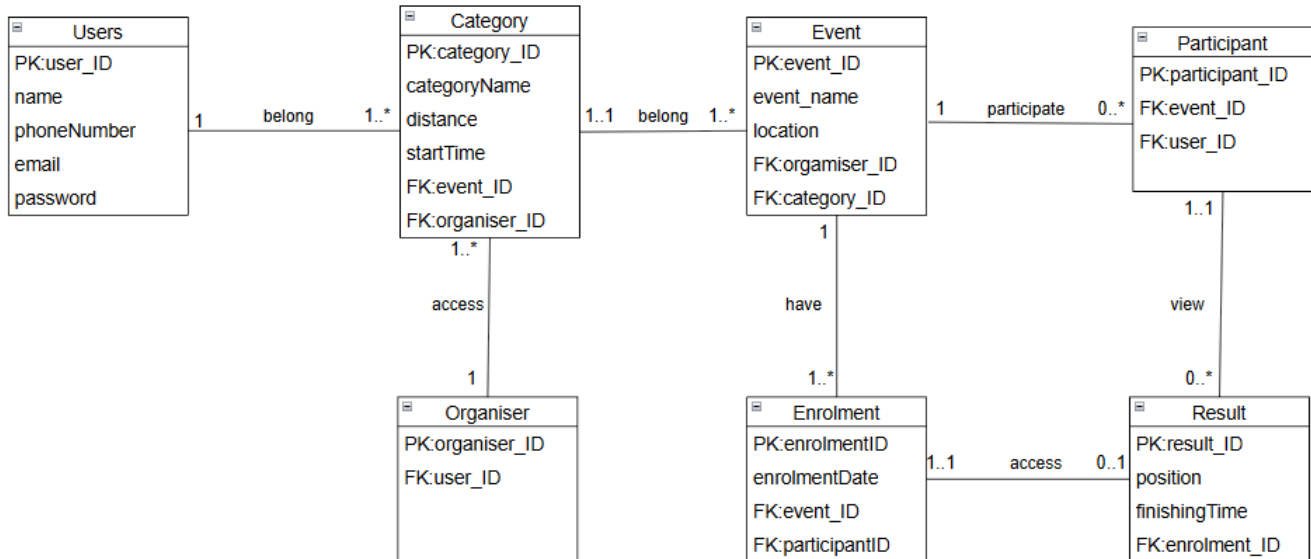
St10465462

Prog6212

POE Part 1

Section A

ERD



Section B

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either a Participant or Organiser.	Public	{“name”, “email”, “password”, “role”}	201 Created – user registered. 400 Bad Request – invalid details.
POST	/api/auth/login	Authenticates a registered user and provide access based on their role.	Public	{ “email”, “password” }	200 OK – login successful. 401 Unauthorised – invalid credentials.
GET	/api/profile	Allows a logged in user to view their own profile information.	Participant / Organiser	None	200 OK – profile details. 401 Unauthorised – not logged in.
PUT	/api/profile	Allows a logged in user to update their own profile information.	Participant / Organiser	Updated profile details	200 OK – profile updated. 400 Bad Request – invalid details.
GET	/api/events	Returns a list of available events that users can browse.	Participant / Organiser	Return Events	200 OK –list of events.
GET	/api/events/{id}	Return the details of a specific event.	Participant / Organiser	Return details of event	200 OK – event details. 404 Not Found – event does not exist.
POST	/api/events	Creates a new event containing its name, description, date, location, distance and event type.	Organiser	{“name”, “description”, “date”, “location”, “distance”, “eventType”}	201 Created – event created. 400 Bad Request – invalid data.
PUT	/api/events/{id}	Updates an event managed by the organiser.	Organiser	Updated event details	200 OK – event updated. 404 Not Found – event does not exist.
DELETE	/api/events/{id}	Deletes an event managed by the organiser.	Organiser	Delete	204 No Content – event deleted. 404 Not Found – event does not exist.
GET	/api/events/{eventId}/categories	Allows users to view the categories available for a specific event.	Participant/ Organiser	View category	200 OK –list of categories. 404 Not Found – event does not exist.
POST	/api/events/{eventId}/categories	Creates an age or distance category for an event.	Organiser	{“name”, “type”}	201 Created – category created. 400 Bad Request – invalid data.
PUT	/api/categories/{id}	Updates an existing category for an event.	Organiser	Updated category details	200 OK – category updated. 404 Not Found – category does not exist
DELETE	/api/categories/{id}	Delete a category from an event.	Organiser	Delete category id	204 No Content – category deleted. 404 Not Found – category does not exist

POST	/api/events/{eventId}/registrations	Allows a participant to enter an event by selecting a category.	Participant	{"categoryId"}	201 Created – registration created. 400 Bad Request – invalid category/event.
GET	/api/events/{eventId}/registrations	Allows an organiser to view all participant registrations for their event.	Organiser	View registration	200 OK – list of registrations. 404 Not Found – event does not exist.
POST	/api/registrations/{registrationId}/result	Records a participants finishing time and finishing position after an event.	Organiser	{"finishingTime", "finishingPosition"}	201 Created – result recorded. 400 Bad Request – invalid result.
GET	/api/result/me	Allows a participant to view their own race results and performance history.	Participant	View race result	200 OK – participant result. 404 Not Found – no results found.

Section C

```
CREATE DATABASE RaceDay;
USE RaceDay;

CREATE TABLE Users(
  user_ID INT IDENTITY PRIMARY KEY,
  name VARCHAR(20),
  phoneNumber INT,
  email VARCHAR(30),
  password VARCHAR(30)
);

--Creating table organiser
CREATE TABLE Organiser(
  organiser_ID INT IDENTITY(1,1) PRIMARY KEY,
  user_ID INT NOT NULL,
  FOREIGN KEY (user_ID) REFERENCES Users(user_ID)
);

--Creating table category
CREATE TABLE Category(
  category_ID INT IDENTITY(1,1) PRIMARY KEY,
  categoryName VARCHAR(50) NOT NULL,
  distance DECIMAL(10,2),
  StartTime TIME,
  event_ID INT NULL,
  organiser_ID INT NULL,
  FOREIGN KEY (organiser_ID) REFERENCES Organiser(organiser_ID)
);

--Creating table event
CREATE TABLE Event(
  event_ID INT IDENTITY(1,1) PRIMARY KEY,
  event_name VARCHAR(100) NOT NULL,
  location VARCHAR(100) NOT NULL,
  organiser_ID INT NOT NULL,
  category_ID INT NOT NULL,
  FOREIGN KEY (organiser_ID) REFERENCES Organiser(organiser_ID),
  FOREIGN KEY (category_ID) REFERENCES Category(category_ID)
);

--Creating table Participant
CREATE TABLE Participant(
  participant_ID INT IDENTITY(1,1) PRIMARY KEY,
  event_ID INT NOT NULL,
  user_ID INT NOT NULL,
  FOREIGN KEY (event_ID) REFERENCES Event(event_ID),
  FOREIGN KEY (user_ID) REFERENCES Users(user_ID)
);

--Creating table enrolment
CREATE TABLE Enrolment(
  enrolmentID INT IDENTITY(1,1) PRIMARY KEY,
  enrolmentDate DATE NOT NULL,
  event_ID INT NOT NULL,
  participantID INT NOT NULL,
  FOREIGN KEY (event_ID) REFERENCES Event(event_ID),
  FOREIGN KEY (participantID) REFERENCES Participant(participant_ID)
);
```

```

-- Creating the Result table
CREATE TABLE Result(
result_ID INT IDENTITY(1,1) PRIMARY KEY,
position INT,
finishingTime TIME,
enrolment_ID INT NOT NULL,
FOREIGN KEY (enrolment_ID) REFERENCES Enrolment(enrolmentID)
);

-- Inserting users
INSERT INTO Users (name, phoneNumber, email, password)
VALUES
('Siyanda Ndlovu', '0712345678', 'siyanda@gmail.com', 'Pass123'),
('Thando Mkhize', '0723456789', 'thando@gmail.com', 'Pass456'),
('Lerato Dlamini', '0734567890', 'lerato@gmail.com', 'Pass789'),
('Musa Nkosi', '0745678901', 'musa@gmail.com', 'Pass321'),
('Kabelo Mokoena', '0756789012', 'kabelo@gmail.com', 'Pass654');

SELECT * FROM Users;

-- Inserting organisers
INSERT INTO Organiser (user_ID)
VALUES
(1),
(2),
(3);

SELECT * FROM Organiser;

-- Inserting participants
INSERT INTO Participant (event_ID, user_ID)
VALUES
(1, 4),
(1, 5),
(2, 1),
(2, 3),
(3, 2);

SELECT * FROM Participant;

-- Inserting events
INSERT INTO Event
(event_name, location, organiser_ID, category_ID)
VALUES
('Johannesburg Fun Run', 'Johannesburg', 1, 1),
('Durban Road Race', 'Durban', 2, 2),
('Cape Town Half Marathon', 'Cape Town', 3, 3);

SELECT * FROM Event;

-- Inserting categories
INSERT INTO Category
(categoryName, distance, StartTime, event_ID, organiser_ID)
VALUES
('5 KM Fun Run', 5.00, '07:00', NULL, 1),
('10 KM Road Race', 10.00, '06:30', NULL, 2),
('Half Marathon', 21.10, '06:00', NULL, 3);

SELECT * FROM Category;

-- Inserting enrolments
INSERT INTO Enrolment
(enrolmentDate, event_ID, participantID)
VALUES
('2026-08-01', 1, 1),
('2026-08-02', 1, 2),
('2026-08-03', 2, 3),
('2026-08-04', 2, 4),
('2026-08-05', 3, 5);

SELECT * FROM Enrolment;

```

References

Microsoft, 2026. *CREATE TABLE* (Transact-SQL). [online] Microsoft Learn. Available at: [Microsoft Learn – CREATE TABLE \(Transact-SQL\)](#) [Accessed 4 September 2026].

Microsoft, 2026. Primary and foreign key constraints – SQL Server. [online] Microsoft Learn. Available at: [Microsoft Learn – Primary and foreign key constraints](#) [Accessed 4 September 2026].

Mozilla Developer Network, 2025. POST request method. [online] MDN Web Docs. Available at: [MDN Web Docs – POST request method](#) [Accessed 4 September 2026].